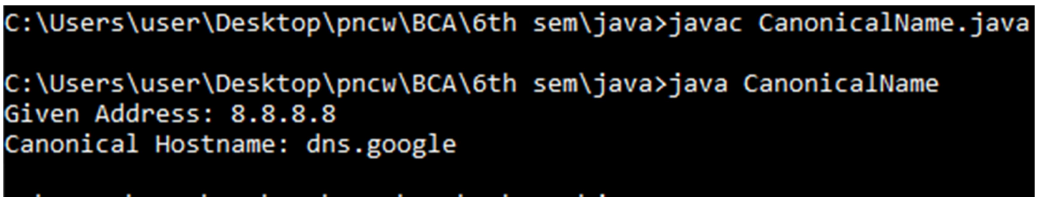
Output

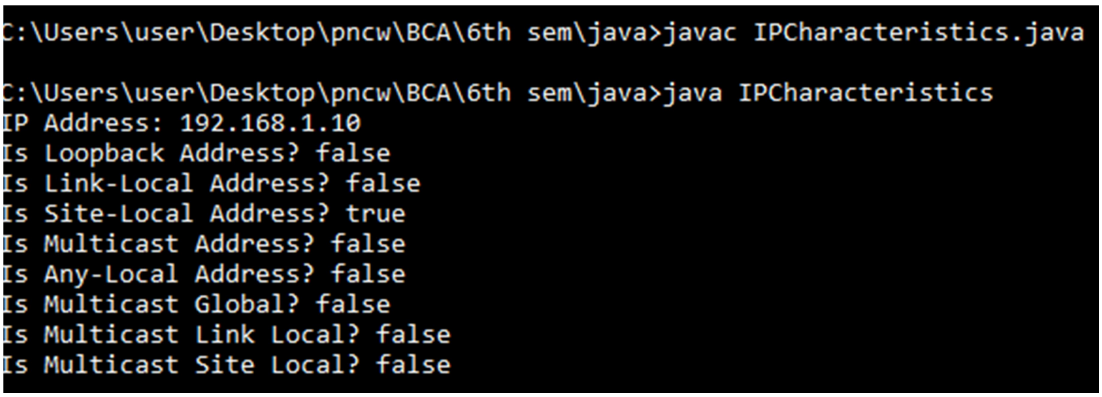
```
Hostname: prod-repl-java-6f59d9477d-sfmsg
IP Address: 10.236.27.4

=== Code Execution Successful ===
```

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>javac RemoteIP.java

C:\Users\user\Desktop\pncw\BCA\6th sem\java>java RemoteIP
Remote Host: google.com
IP Address: 142.250.192.238
```


C:\Users\user\Desktop\pncw\BCA\6th sem\java>javac CanonicalName.java
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java CanonicalName
Given Address: 8.8.8.8
Canonical Hostname: dns.google

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>javac IPVersions.java

C:\Users\user\Desktop\pncw\BCA\6th sem\java>java IPVersions
IP addresses for: google.com
IPv4: 142.250.192.238
IPv6: 2404:6800:4002:81c:0:0:0:200e

C:\Users\user\Desktop\pncw\BCA\6th sem\java>
```

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>javac CheckIPVersion.java

C:\Users\user\Desktop\pncw\BCA\6th sem\java>java CheckIPVersion
Input Address: 2001:4860:4860:0:0:0:0:8888
The address is IPv6

C:\Users\user\Desktop\pncw\BCA\6th sem\java>
```


C:\Users\user\Desktop\pncw\BCA\6th sem\java>javac IPCharacteristics.java
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java IPCharacteristics
IP Address: 192.168.1.10
Is Loopback Address? false
Is Link-Local Address? false
Is Site-Local Address? true
Is Multicast Address? false
Is Any-Local Address? false
Is Multicast Global? false
Is Multicast Link Local? false
Is Multicast Site Local? false

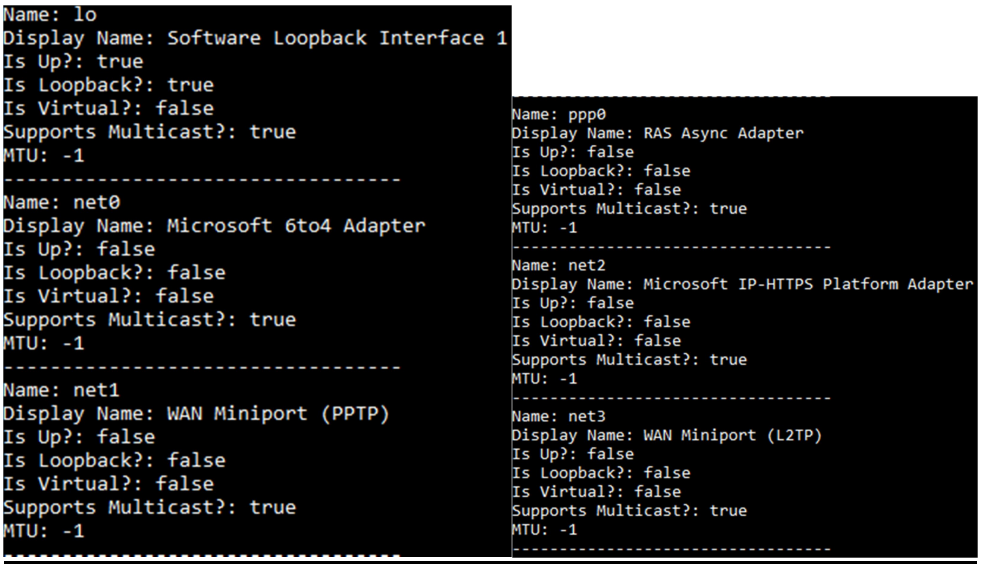
```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java CompareDomains
Error: java.net.UnknownHostException: No such host is known (helios.ibiblio.org)

C:\Users\user\Desktop\pncw\BCA\6th sem\java>
```

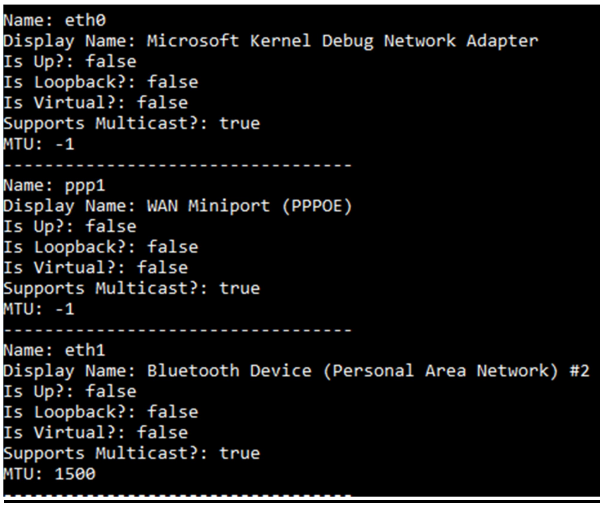
Conclusion

Observation

```
Interface: lo
   Address: 127.0.0.1
   Address: 0:0:0:0:0:0:0:1

Interface: net0

Interface: net1

Interface: ppp0

Interface: net2

Interface: net3

Interface: eth0

Interface: ppp1

Interface: eth1
   Address: fe80:0:0:0:d177:ff2e:b838:c6a3%eth1

Interface: eth2

Interface: wlan0
   Address: 192.168.1.107
   Address: 2400:1a00:5b29:7e5c:d603:7336:bece:2358
   Address: 2400:1a00:5b29:7e5c:549c:459a:5b66:ed65
   Address: fe80:0:0:0:26b2:9245:92b2:5cd4%wlan0

Interface: wlan1

Interface: wlan2

Interface: eth3

Interface: net4

Interface: wlan3
   Address: fe80:0:0:0:66ec:3fe8:e09:3b20%wlan3
```

Conclusion


Name: lo
Display Name: Software Loopback Interface 1
Is Up?: true
Is Loopback?: true
Is Virtual?: false
Supports Multicast?: true
MTU: -1
Name: net0
Display Name: Microsoft 6to4 Adapter
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
Name: net1
Display Name: WAN Miniport (PPTP)
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
Name: ppp0
Display Name: RAS Async Adapter
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
Name: net2
Display Name: Microsoft IP-HTTPS Platform Adapter
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
Name: net3
Display Name: WAN Miniport (L2TP)
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1


Name: eth0
Display Name: Microsoft Kernel Debug Network Adapter
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
Name: ppp1
Display Name: WAN Miniport (PPPOE)
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
Name: eth1
Display Name: Bluetooth Device (Personal Area Network) #2
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: 1500

Conclusion

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java CheckReachability
Pinging: www.google.com
Host is reachable.
```

Conclusion

The program successfully demonstrated how to check if a remote host is reachable using Java's isReachable() method. This is useful for basic network diagnostics and testing connectivity between systems.

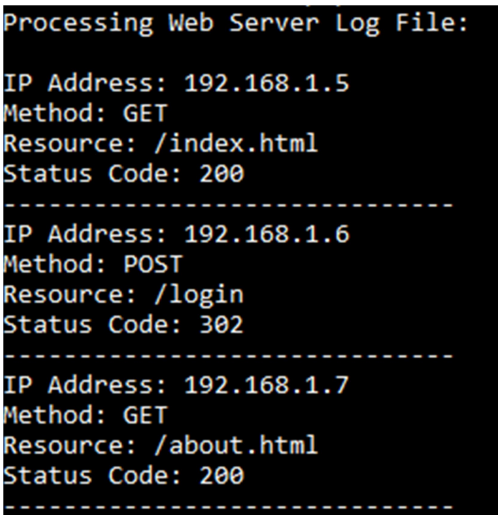
Observation

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java SpamCheck
Enter a message:
Click here to win free macbook.
This message is SPAM.
```

Conclusion

The program successfully demonstrated a basic spam detection mechanism using keyword matching. It shows how simple text analysis can be used to identify suspicious or unwanted messages.

Observation



Conclusion

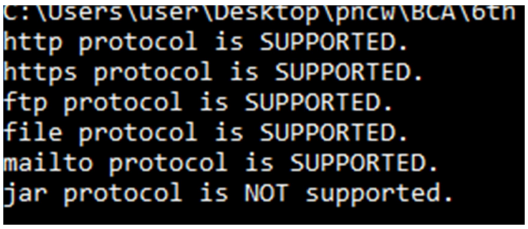
Java's file-handling features make it easy to process server log files. The program demonstrated how to read log entries, split them into components, and analyze useful information for monitoring server performance.

Observation

```
Full URL: https://www.example.com:8080/path/to/resource?query=test
Protocol: https
Host: www.example.com
Port: 8080
Path: /path/to/resource
Query: query=test
File: /path/to/resource?query=test
```

Conclusion

By using the URL class and its getter methods, we can easily extract individual parts of a URL. This demonstrates how Java provides built-in functionality for URL parsing without manually splitting strings.



Conclusion

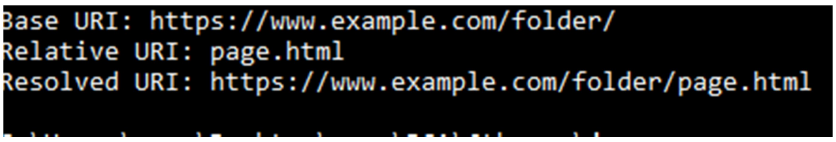
By using the URL class and handling MalformedURLException, we can check which protocols the virtual machine supports. This method provides a simple way to test network protocol handling in Java.

Observation

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>javac DownloadWebPage.java

C:\Users\user\Desktop\pncw\BCA\6th sem\java>java DownloadWebPage
Web page downloaded successfully!
```

Conclusion

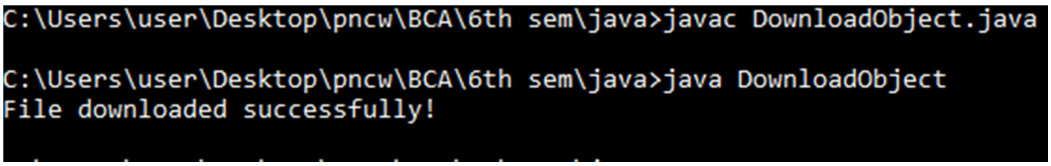
Using Java's URL and URLConnection classes, we can download a web page's content and save it locally. This demonstrates basic web scraping and network programming in Java.



Conclusion

The program demonstrates how Java's URI class can resolve relative URIs to absolute URIs using the resolve() method, which is helpful in web development and resource management.

Observation



Conclusion

By using Java's URL and openStream() methods along with FileOutputStream, we can download objects or files from the internet efficiently. This demonstrates basic file handling and network programming in Java.

```
C:\Users\user\Desktop\pnCW\BCA\6th sem\java>java XWWWFormEncoded
x-www-form-urlencoded string:
name=Deep&email=deep.drsn%40example.com&message=Hello%2C+how+are+you%3F
```

Conclusion

The program demonstrates how Java's URLEncoder class converts strings into x-www-form-urlencoded format. This is essential for sending safe and properly encoded form data over HTTP.

Observation

Conclusion

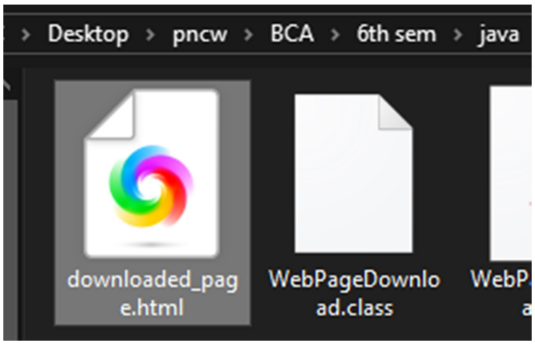
By using Java's URL and HttpURLConnection classes, we can communicate with server-side programs using HTTP GET. This is useful for fetching data or interacting with web services.

31.			
32.			
33.			
34.			
35.			

Observation

When the program is executed, it connects to the given URL, reads all the HTML content, and stores it in a file named downloaded_page.html.

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>javac WebPageDownload.java

C:\Users\user\Desktop\pncw\BCA\6th sem\java>java WebPageDownload
Web page downloaded successfully!
```



Conclusion

The program successfully demonstrates how to use URLConnection to access a webpage and download its content. This shows how Java can be used for network programming tasks such as retrieving data from the internet.

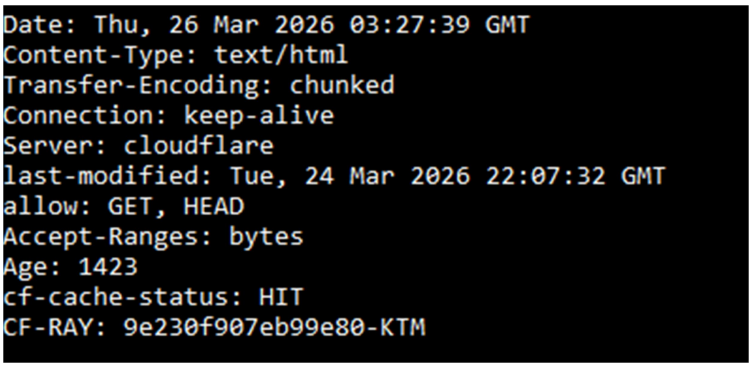
Observation

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java ReadHTTPHeaders
HTTP Header Fields:

Date: Thu, 26 Mar 2026 03:09:55 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: keep-alive
Server: cloudflare
last-modified: Tue, 24 Mar 2026 22:07:32 GMT
allow: GET, HEAD
Accept-Ranges: bytes
Age: 359
cf-cache-status: HIT
CF-RAY: 9e22f593cd5a98d6-KTM
```

Conclusion

The program successfully retrieves and prints all HTTP header fields from a webpage using HttpURLConnection. This demonstrates how Java can interact with HTTP protocol metadata in network programming.

Observation



Conclusion

The program successfully prints all HTTP header fields using HttpURLConnection. This demonstrates how Java applications can read and process HTTP protocol metadata, which is essential in network programming

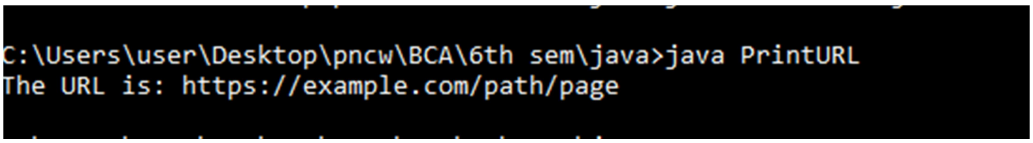
Observation

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java HttpRequestMethod
HTTP Request Method Used: GET
```

Conclusion

The program successfully retrieves and displays the HTTP request method using HttpURLConnection. This demonstrates how Java handles HTTP communication and how request types are identified in network programming.

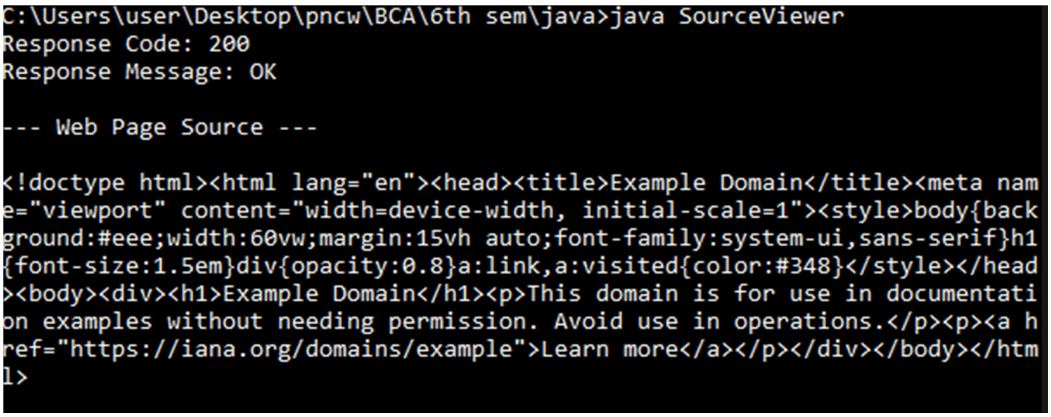
Observation



Conclusion

The program successfully demonstrates how to create a URL object and print it in Java. This shows the basic usage of the URL class in network programming.

Observation



Conclusion

The program successfully acts as a source viewer that retrieves and displays both the HTTP response details and the full webpage source. This demonstrates how Java handles server communication, response checking, and reading webpage content.

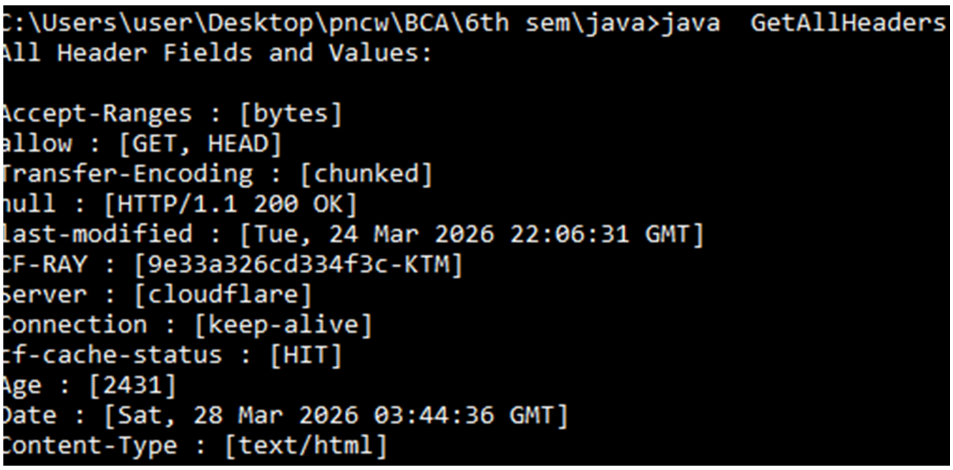
Observation

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>javac LastModifiedTime.java

C:\Users\user\Desktop\pncw\BCA\6th sem\java>java LastModifiedTime
Last Modified: Wed Mar 25 03:51:31 NPT 2026
```

Conclusion

The program successfully retrieves the Last-Modified timestamp of a URL using Java's URLConnection. This helps understand how web servers share metadata about resource updates in network programming.

Observation



Conclusion

The program successfully retrieves and displays all HTTP header fields and their values using Java's URLConnection. This demonstrates how clients can access important metadata sent by web servers

Observation

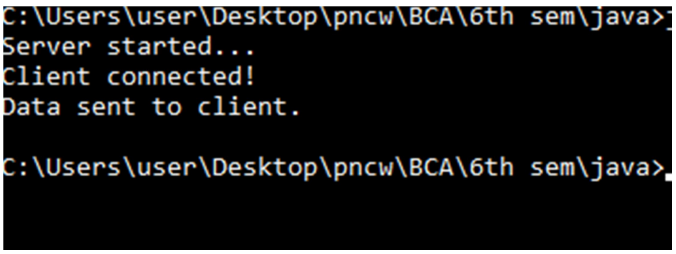
```
--- Server Response ---

HTTP/1.1 200 OK
Date: Sat, 28 Mar 2026 04:16:55 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
Server: cloudflare
Last-Modified: Tue, 24 Mar 2026 22:06:31 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 5271
cf-cache-status: HIT
CF-RAY: 9e33d277ca037674-KTM

210
<!doctype html><html lang="en"><head><title>Example Domain</t
itle><meta name="viewport" content="width=device-width, initi
al-scale=1"><style>body{background:#eee;width:60vw;margin:15v
h auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}di
v{opacity:0.8}a:link,a:visited{color:#348}</style></head><bod
y><div><h1>Example Domain</h1><p>This domain is for use in do
cumentation examples without needing permission. Avoid use in
 operations.</p><p><a href="https://iana.org/domains/example"
>Learn more</a></p></div></body></html>

0
```

Conclusion

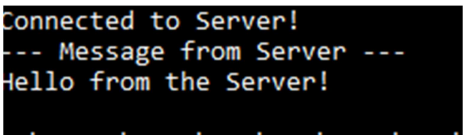
The program successfully reads data from a server using Java's Socket class. It demonstrates low-level network communication, showing how a client can manually send requests and receive responses directly from a server.

Observation

Server



Client



Conclusion

This program demonstrates how a server can write/send data to a client using Java Socket programming. The server uses ServerSocket to listen for connections and PrintWriter to send messages. This lab shows the basics of client-server communication in networking.

Observation

Client

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java Client
Time sent to server: 13:10:30.999372600

C:\Users\user\Desktop\pncw\BCA\6th sem\java>
```

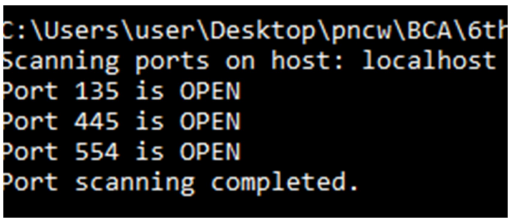
Server

```
Server is running...
Client connected.
Time received from client: 13:10:30.999372600
```

Conclusion

The program successfully demonstrates socket communication in Java. The client is able to send its current time to the server, and the server reads and displays it. This exercise helps understand network programming basics, client-server communication, and the use of Input/Output streams with sockets.

Observation



Conclusion

The program successfully scans low-numbered ports using Java sockets. It demonstrates how to, create socket connections, detect open ports, and practical use of socket exceptions. This experiment helps understand basic security concepts and how servers expose services on specific ports.

Observation

Conclusion

The program successfully retrieves and displays detailed socket information. This lab helps in understanding how sockets connect between two endpoints, how to extract socket metadata, the concept of local versus remote ports, and how to check the connection status.

Observation

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java sServer
Server started... waiting for client.
Client connected from: /127.0.0.1
```

Conclusion

The program successfully demonstrates how to create a simple server socket in Java. It shows how to open a listening port, how to accept client connections, and how the server terminates after establishing a connection.

```
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Observation

Server

```
C:\Users\user\Desktop\pncw\BCA\6th
DateTime Server started...
Client connected: /127.0.0.1
```

Client

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java DateTimeClient
From Server: Current Date & Time: 2026-03-29T09:18:52.428120200
```

Conclusion

The program successfully demonstrates how to use multi-threading with sockets to create a server that can serve multiple clients at the same time. Each client receives the current system date and time independently